Stabilization Demo Document for SDET-Retail-App using Playwright

App: SDET-Retail-App
Author: Justin Philip
Date: 04/07/2026
Environment: Local System
Browser: Chrome

Test Report

Test Environment:

Application: SDET Retail-Application

Test Automation Framework: Playwright

Browser: Chromium

Environment Type: Local Development Environment

Test Objective

To evaluate application resilience and test stability under unstable network conditions, asynchronous processing delays, and timing-related race conditions.

Simulation Techniques Used

- Network request interception
- Artificial API delays
- Request failures
- Random response behavior
- Repeated test execution

Validation Strategy:

```
C:\Users\[redacted]\My files\PLAYWRIGHT_WORKS\SDET\SDET_Week5>npx playwright test tes
ts/time-race.spec.ts --repeat-each=50
```

The same test is executed multiple times to identify intermittent failures and verify stabilization after implementing corrective measures.

Scenario 1 – Flaky Test Caused by Network Instability During Sales Synchronization

Test Objective:
Validate that a newly queued sale is successfully synchronized and reflected in the Local Outbox.

Failure Simulation Strategy:
To simulate real-world network instability, the sales API endpoint was intercepted and manipulated using Playwright routing.

The following behaviors were introduced:

- 25% Probability: API request aborted.
- 25% Probability: API response delayed.
- 50% Probability: API responds normally.

```
await page.route("**/api/sales", async (route)=>{
        const random = Math.random();
        if(random<0.25){
          log.warn("Request is aborted, due to unexpected circumstances");
            route.abort('failed');
            console.log(random);
        }else if(random<0.50){
          const delay = 100 + Math.random() * 100;
          log.warn(`API is slow; Expected Delay ${delay} ms`)
          await new Promise(resolve => setTimeout(resolve, delay));
          await route.continue();
        }else{
          log.warn("Simulated correct API response");
          await route.continue();
          console.log(random);
        }
      })
```

This simulation reproduced intermittent network conditions that users may experience in production environments.

Test Scenario

Steps Performed

1. Open POS page.
2. Verify application is online.
3. Reset local outbox.
4. Confirm no queued sales exist.
5. Queue a new sale.

6. Verify synchronization completion.

Symptom

The test intermittently failed while validating the synchronization status of a newly queued sale.

Failure Message
Error: expect(locator).toHaveText(expected) failed

Locator: getByTestId('outbox-status')
Expected: "synced"
Error: element(s) not found

Observed Behavior

During failed executions:

- Sale was queued successfully.
- Synchronization request was either delayed or aborted.
- Synchronization status element never appeared.
- Assertion failed while waiting for the status.

Evidence

Figure 1 – Overall Test Execution Report: Playwright execution report showing intermittent failures during repeated execution.

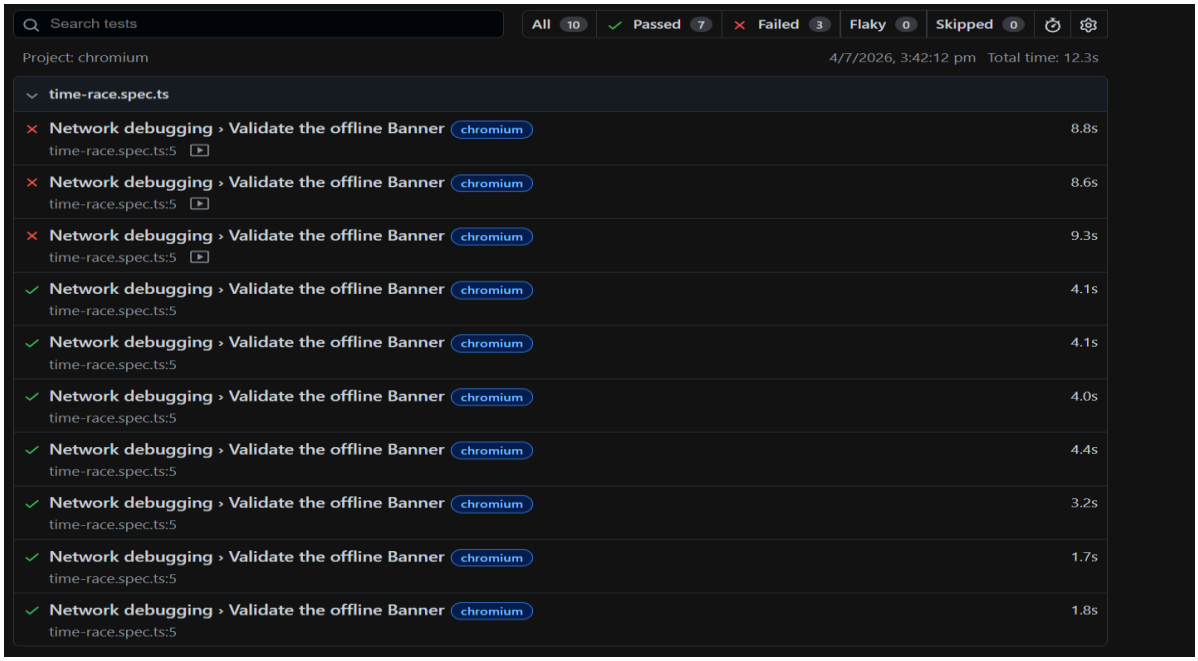


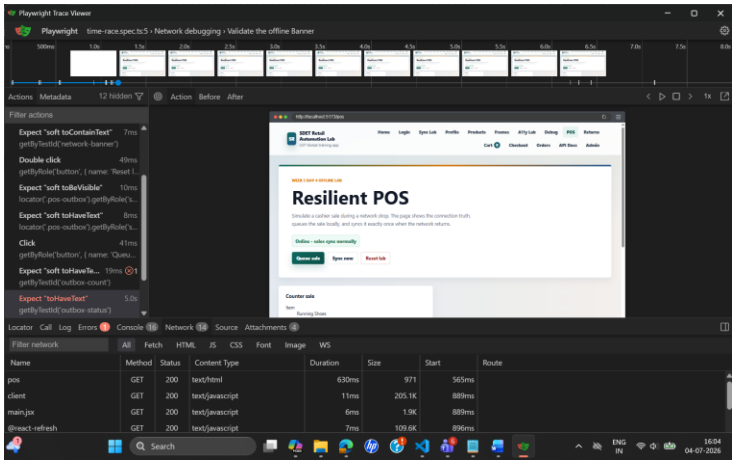
Figure 2 – Failed Assertion

Figure 2 – Application state captured during test failure. Synchronization status element was either not synchronized or not present when the assertion executed.

```
Error: expect(locator).toHaveText(expected) failed

Locator:  getByTestId('outbox-count')
Expected: "0"
Received: "1"
Timeout:  250ms

Call log:
  - Expect "soft toHaveText" with timeout 250ms
  - waiting for getByTestId('outbox-count')
    5 × locator resolved to <strong data-testid="outbox-count">1</strong>
      - unexpected value "1"


  48 |
  49 |       log.info("Pending counts increase is being verified")
> 50 |       await expect.soft(page.getByTestId("outbox-count")).toHaveText("0", { timeout: 250 });
     |                                                           ^
  51 |
  52 |       log.info("New sale is being synced...")
  53 |       await expect(page.getByTestId("outbox-status")).toHaveText("synced");
    at C:\Users\310973\My files\PLAYWRIGHT_WORKS\SDET\SDET_Week5\tests\time-race.spec.ts:50:59
```

```
Errors

Error: expect(locator).toHaveText(expected) failed

Locator: getByTestId('outbox-status')
Expected: "synced"
Timeout: 5000ms
Error: element(s) not found

Call log:
  - Expect "toHaveText" with timeout 5000ms
  - waiting for getByTestId('outbox-status')


  50 |
  51 |       log.info("New sale is being synced...")
> 52 |       await expect(page.getByTestId("outbox-status")).toHaveText("synced");
     |                                                       ^
  53 |
  54 |
  55 |
    at C:\Users\310973\My files\PLAYWRIGHT_WORKS\SDET\SDET_Week5\tests\time-race.spec.ts:52:55
```

Figure 3 – Playwright Trace Timeline showing the sequence of actions leading to the synchronization failure.



Root Cause Analysis

The synchronization process relies on a successful response from:

/api/sales

The test intentionally introduced unstable network behavior.

Execution Flow During Failure

Queue Sale
↓
POST /api/sales
↓
Network Delay / Request Abort
↓
Synchronization Incomplete
↓
outbox-status Element Not Created
↓
Test Assertion Executes
↓
Failure

Fix Implemented

The test was using an aggressive timeout and validating the final synchronization state before the synchronization workflow completed.

Before

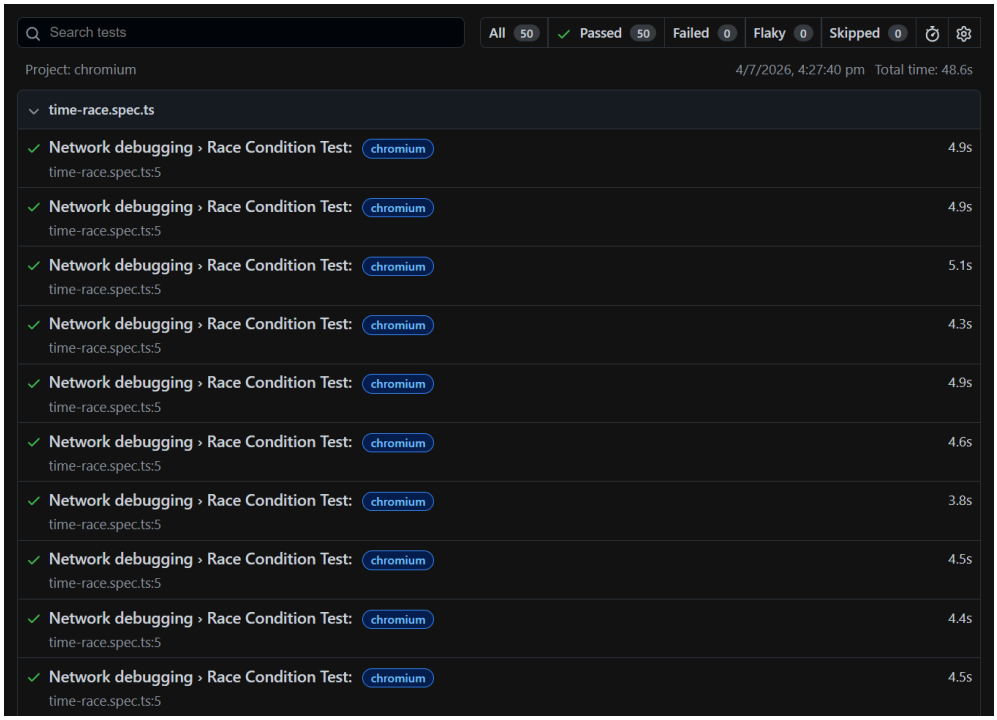
```
log.info("Pending counts increase is being verified");
    await expect.soft(page.getByTestId("outbox-count")).toHaveText("0", { timeout: 250 });

    log.info("New sale is being synced...");
    await expect(page.getByTestId("outbox-status")).toHaveText("synced", {timeout:
250});
```

After

```
log.info("Pending counts increase is being verified");
    await expect.soft(page.getByTestId("outbox-count")).toHaveText("0", { timeout: 10000
});

    log.info("New sale is being synced...");
    await expect(page.getByTestId("outbox-status")).toHaveText("synced", {timeout:
10000});
```

Proof:

Running 50x for flakiness



Scenario 2 – Currency Precision Defect Caused by Floating-Point Arithmetic

Test Objective

Validate that a partial refund is calculated and displayed accurately using currency values without losing precision.

Failure Simulation Strategy

To simulate a real-world financial calculation defect, the refund amount returned by the Refund API was intentionally modified before being rendered in the UI.

The original refund amount: -> 34965 paise = ₹349.6

was artificially reduced by 1 paisa: -> 34964 paise = ₹349.64

const amountRupees = amountPaise / 100;

const adjustedAmount = parseFloat((amountRupees - 0.01).toFixed(2));

amountPaise = Math.round(adjustedAmount * 100)

Test Scenario

Steps Performed

1. Seed a paid order.
2. Create a partial refund request.
3. Verify refund calculation in the API response.
4. Navigate to the Returns page.
5. Validate displayed refund amount.
6. Validate refund status.

Symptom

The test failed while validating the displayed refund total.

Failure Message

Error: expect(locator).toHaveText(expected) failed

Locator: getByTestId('refund-total')
Expected: "₹349.65"
Received: "₹349.64"
Timeout: 5000ms

Evidence

Figure 1 – Playwright Execution Report:Playwright execution report showing failure in refund amount validation.

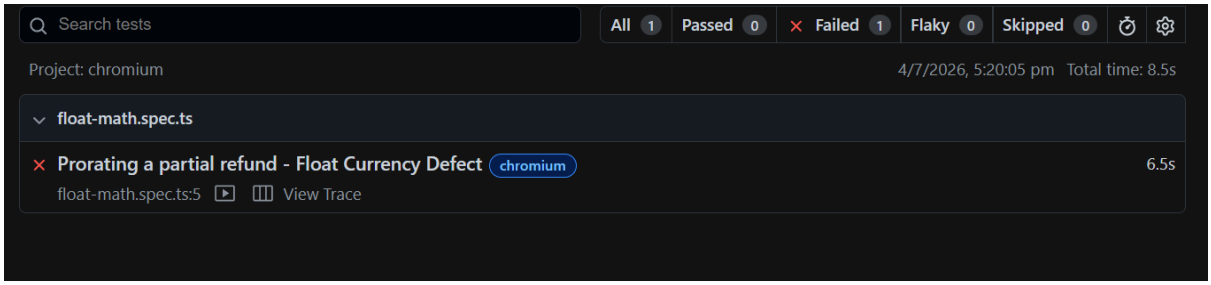
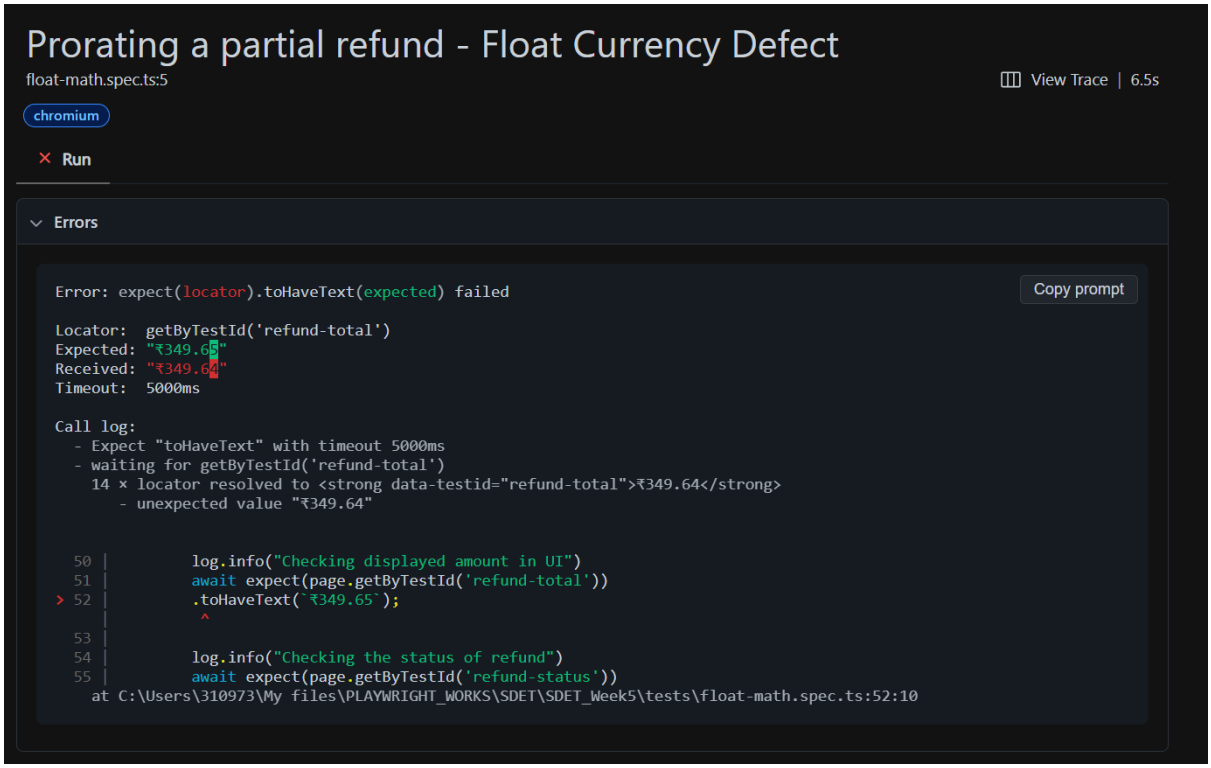


Figure 2 – Refund amount validation failure showing expected and actual values differing by one paisa.



Root Cause Analysis

The refund calculation workflow depends on accurate handling of monetary values.

Expected workflow:
34965 paise
↓
349.65 rupees
↓
Float manipulation
↓
349.64 rupees
↓
Converted back to paise
↓
34964 paise
↓
Displayed to User

Fix Implemented

The defect originated from handling monetary values using floating-point rupee calculations:

Before:

const amountRupees = amountPaise / 100;

const adjustedAmount = parseFloat((amountRupees - 0.01).toFixed(2));

amountPaise = Math.round(adjustedAmount * 100)

After:

const refundAmountPaise = unitPaise * quantity;

const refundAmountRupees = refundAmountPaise / 100;

Proof: